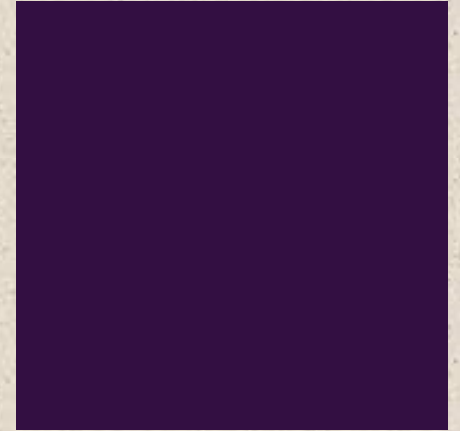
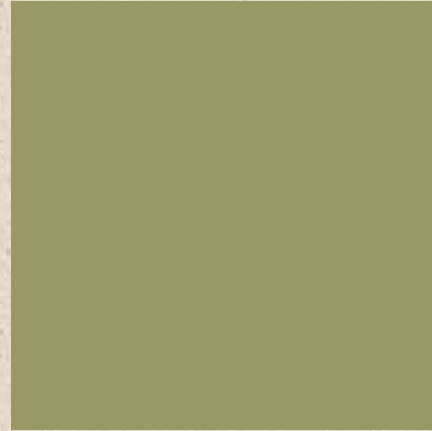
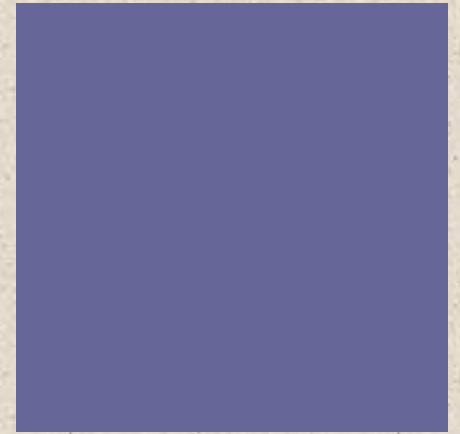
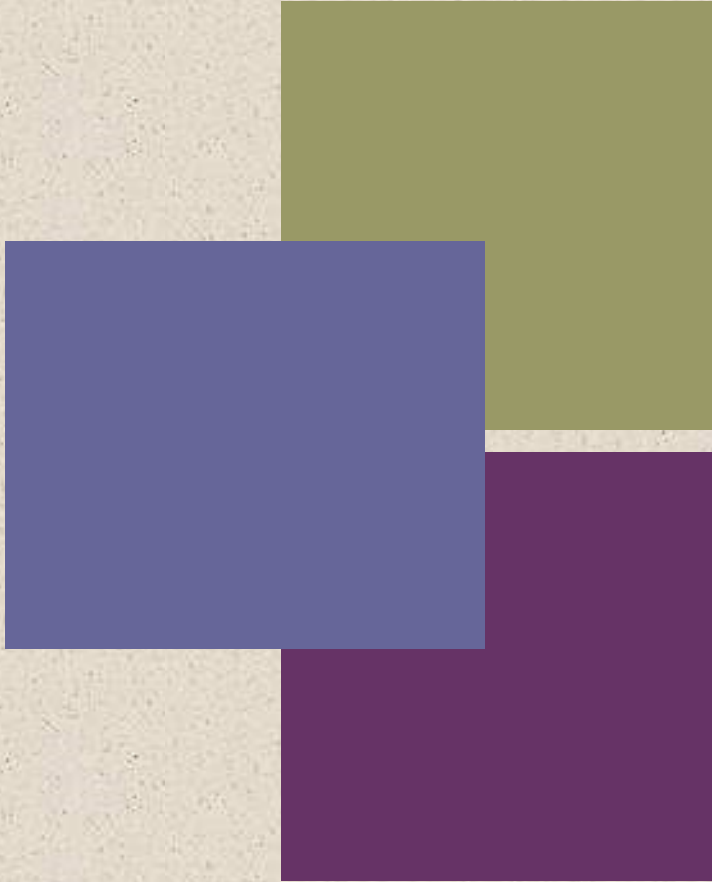




William Stallings Computer Organization and Architecture 10th Edition

Orijinal slaytların
çevirisidir.



+ Chapter 12

Instruction Sets:

Characteristics and Functions

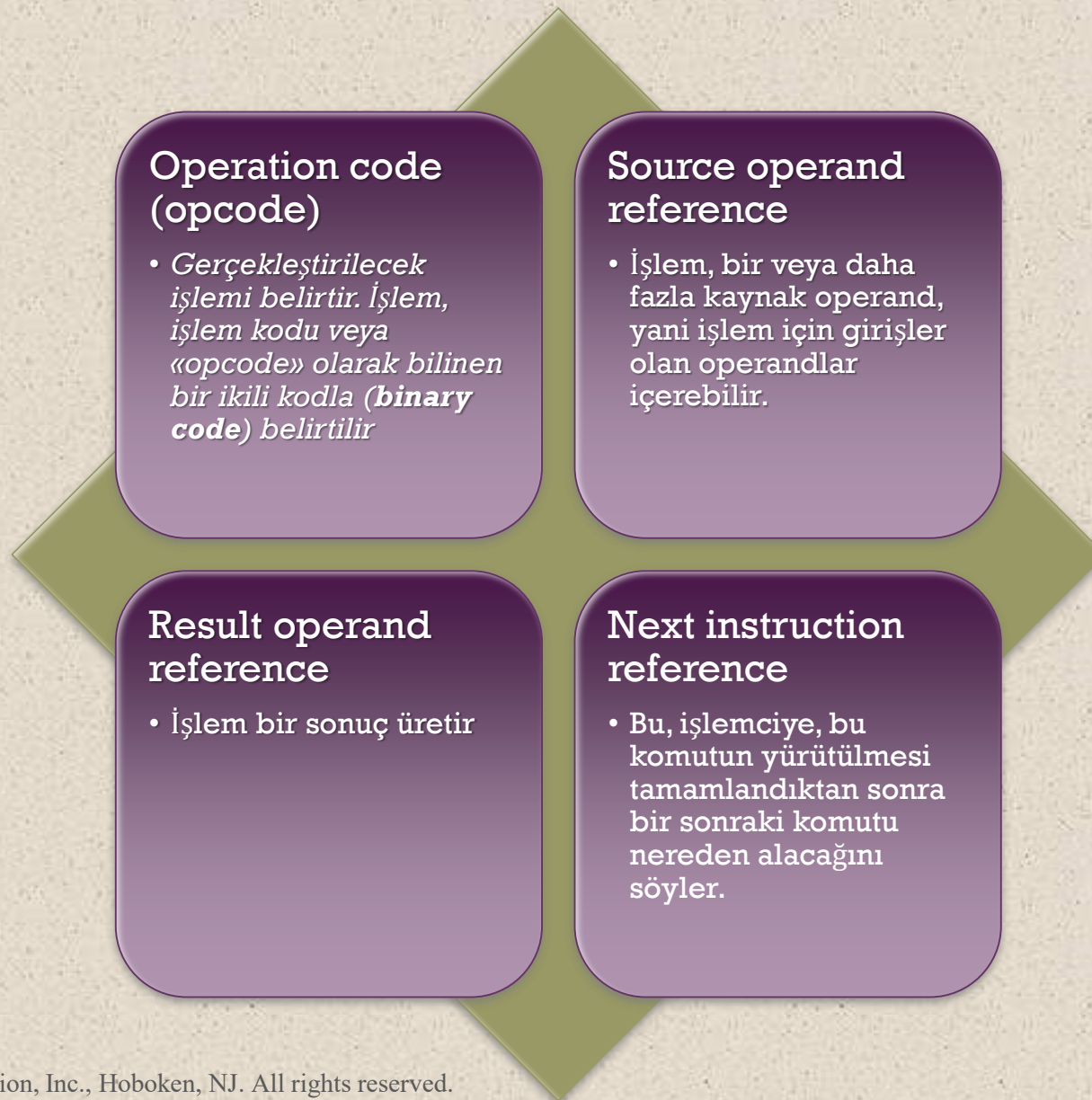


Machine Instruction Characteristics

- İşlemcinin çalışması, makine komutları/emirleri (*machine instructions*) veya bilgisayar komutları (*computer instructions*) olarak adlandırılan, yürüttüğü komutlarla/emirlerle belirlenir.
- İşlemcinin yürütebileceği farklı komutların koleksiyonuna, işlemcinin *komut seti* denir.
- Her komut, işlemcinin yürütme için ihtiyaç duyduğu bilgileri içermelidir.



Elements of a Machine Instruction



Source and result operands can be in one of four areas:

1) Main or virtual memory

- Sıradaki komut referanslarında olduğu gibi, ana veya sanal bellek adresi sağlanmalıdır

2) I/O device

- Komut, işlem için I/O modülünü ve cihazı belirtmelidir. «*Memory mapped*» I/O kullanılıyorsa, bu yalnızca başka bir ana veya sanal bellek adresidir

3) Processor register

- Bir işlemci, makine komutlarıyla referans verilebilecek bir veya daha fazla register içerir.
- Birden fazla register mevcutsa, her bir registera benzersiz bir ad veya numara atanır ve komut, istenen register numarasını içermelidir

4) Immediate

- Operandın değeri, yürütülen komuttaki bir alanda bulunur



Instruction Representation

- Bilgisayarda her komut bir dizi bit ile temsil edilir
- Komut, onu oluşturan unsurlara karşılık gelen alanlara ayrılmıştır.

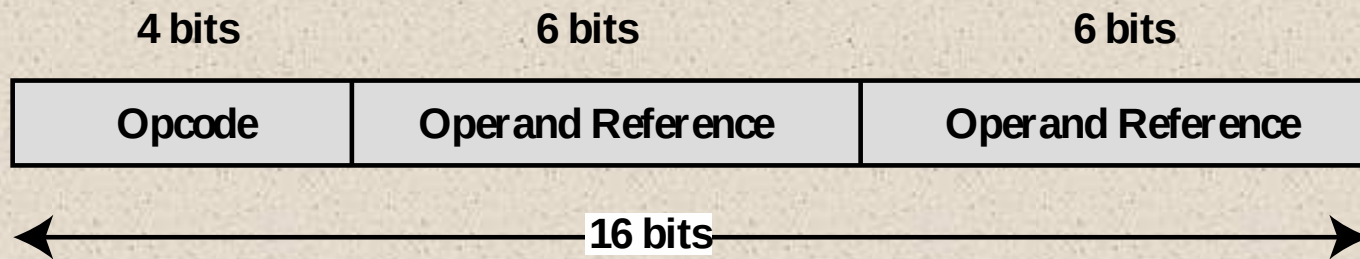


Figure 12.2 A Simple Instruction Format



Instruction Representation



- İşlem kodları, anımsatıcı (*mnemonics*) adı verilen kısaltmalarla temsil edilir
- Examples include:
 - ADD Add
 - SUB Subtract
 - MUL Multiply
 - DIV Divide
 - LOAD Load data from memory
 - STOR Store data to memory
- Operandlar da ayrıca sembolik olarak temsil edilir
- Her sembolik işlem kodunun sabit bir binary gösterimi vardır ve
 - programcı her sembolik operandın konumunu belirtir

Instruction Types

- Aritmetik komutlar, sayısal verileri işlemek için hesaplama yetenekleri sağlar
- Mantık (Boolean) komutları, bir kelimenin bitleri üzerinde sayılar yerine bitler olarak çalışır, bu sayede kullanıcının kullanmak isteyebileceği diğer her türlü veriyi işlemek için yetenekler sağlar.

- Verilerin register ve / veya bellek hücrelerine girip çıkması

Data
processing

Data
storage

Control

Data
movement

- Test komutları, bir veri kelimesinin değerini veya bir hesaplamanın durumunu test etmek için kullanılır.
- Dallar komutları, alınan karara bağlı olarak farklı bir komut dizisine dalar için kullanılır.

- Programları ve verileri belleğe aktarmak ve hesaplamaların sonuçlarını kullanıcıya geri göndermek için I/O komutları gereklidir.



Instruction		Comment
SUB	Y, A, B	$Y \leftarrow A - B$
MPY	T, D, E	$T \leftarrow D \times E$
ADD	T, T, C	$T \leftarrow T + C$
DIV	Y, Y, T	$Y \leftarrow Y \div T$

Üç adres referansını tutmak için nispeten uzun bir komut formatı gerektirdiklerinden, üç adresli komut formatları (three-address instruction formats) yaygın değildir.

(a) Three-address instructions

Instruction		Comment
MOVE	Y, A	$Y \leftarrow A$
SUB	Y, B	$Y \leftarrow Y - B$
MOVE	T, D	$T \leftarrow D$
MPY	T, E	$T \leftarrow T \times E$
ADD	T, C	$T \leftarrow T + C$
DIV	Y, T	$Y \leftarrow Y \div T$

Instruction		Comment
LOAD	D	$AC \leftarrow D$
MPY	E	$AC \leftarrow AC \times E$
ADD	C	$AC \leftarrow AC + C$
STOR	Y	$Y \leftarrow AC$
LOAD	A	$AC \leftarrow A$
SUB	B	$AC \leftarrow AC - B$
DIV	Y	$AC \leftarrow AC \div Y$
STOR	Y	$Y \leftarrow AC$

(b) Two-address instructions

(c) One-address instructions

Tek adresli komut daha basittir. Bunun işe yaraması için ikinci bir adresin imalı/dolaylı/üstü kapalı (**implicit**) olması gerekir. Bu, önceki makinelerde yaygındı, dolaylı adres, akümülatör (**accumulator-AC**) olarak bilinen bir işlemci registeridir.

Figure 12.3 Programs to Execute $Y = \frac{A - B}{C + (D \times E)}$



Table 12.1
Utilization of Instruction Addresses
(Nonbranching Instructions)

Number of Addresses	Symbolic Representation	Interpretation
3	OP A, B, C	$A \leftarrow B \text{ OP } C$
2	OP A, B	$A \leftarrow A \text{ OP } B$
1	OP A	$AC \leftarrow AC \text{ OP } A$
0	OP	$T \leftarrow (T - 1) \text{ OP } T$

AC = accumulator

T = top of stack

(T - 1) = second element of stack

A, B, C = memory or register locations

Instruction Set Design

Çok karmaşık çünkü bilgisayar sisteminin birçok yönünü etkiliyor

İşlemci tarafından gerçekleştirilen birçok fonksiyonu tanımlar

Programcının işlemciyi kontrol etme yolu

Temel tasarım meseleleri:

Operation repertoire

- Kaç tane ve hangi işlemlerin sağlanacağı ve ne kadar karmaşık işlemlerin olması gerektiği

Data types

- Üzerinde işlemlerin gerçekleştirildiği çeşitli veri türleri

Instruction format

- Bit cinsinden komut uzunluğu, adres sayısı, çeşitli alanların boyutu vb.

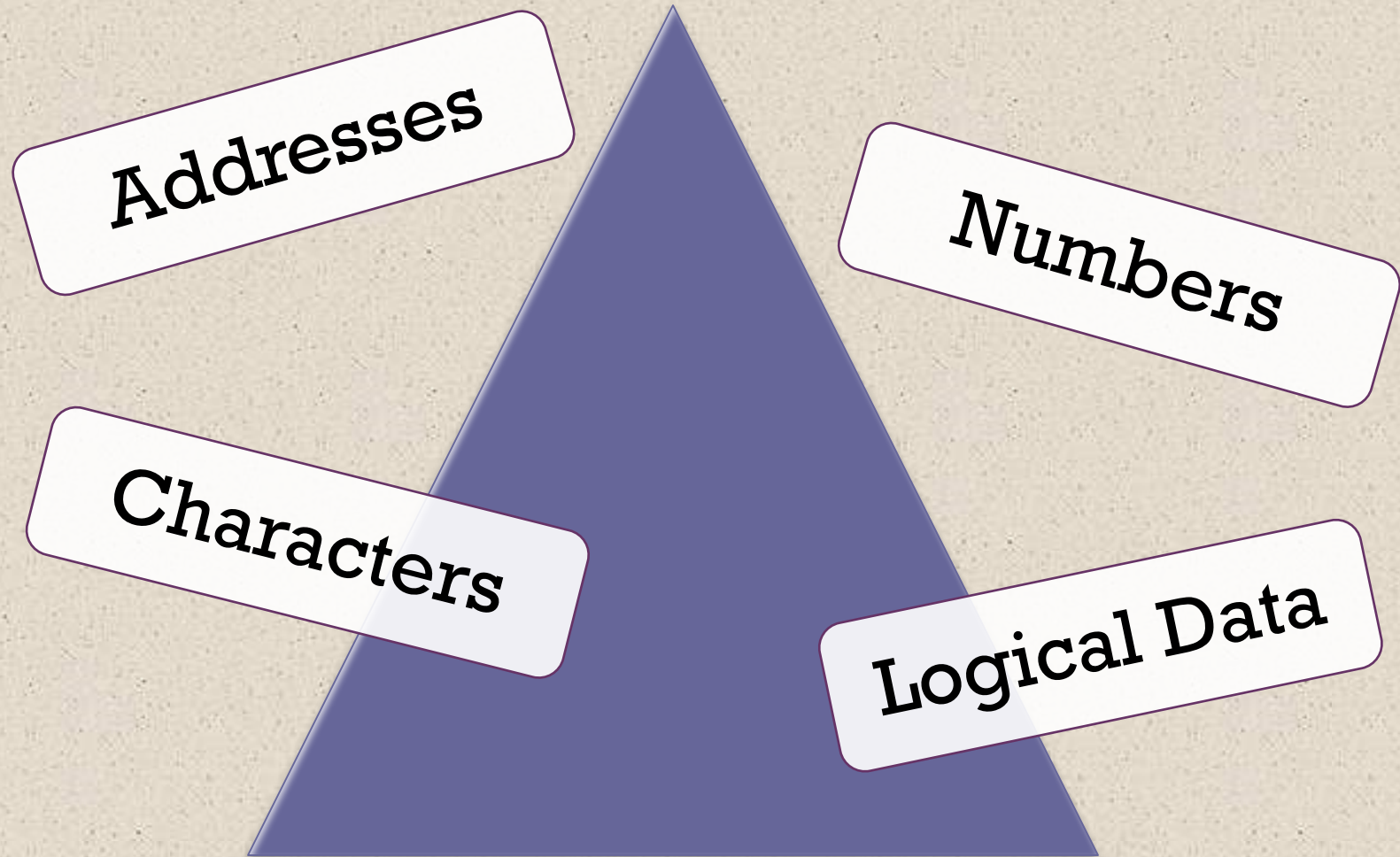
Registers

- Komutlar ve onların kullanımları ile başvurulabilecek işlemci registerlar'ının sayısı

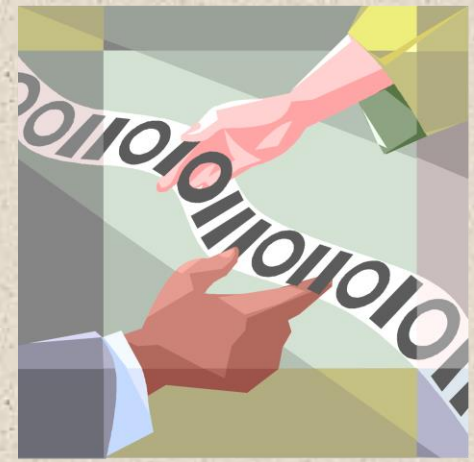
Addressing

- Bir operandın adresinin belirlendiği mod veya modlar

Types of Operands



+ Numbers



- Tüm makine dilleri sayısal veri türlerini içerir
- Bir bilgisayarda saklanan sayılar sınırlıdır:
 - Bir makinede gösterilebilen sayıların büyüklüğünü (*magnitude*) sınırı
 - Kayan noktalı sayılar söz konusu olduğunda, hassasiyetlerinin (*precision*) bir sınırı
- Bilgisayarlarda üç tür sayısal veri yaygındır:
 - Binary integer or binary fixed point
 - Binary floating point
 - Decimal
- Packed decimal
 - Her ondalık basamak (decimal digit), bayt başına depolanan iki basamaklı 4 bitlik bir kodla temsil edilir
 - Sayıları oluşturmak için 4 bitlik kodlar, genellikle 8 bitin katları halinde dizilir.

Tüm dahili bilgisayar işlemleri doğası gereği ikili olmasına rağmen, sistemin insan kullanıcılar ondalık sayılarla ilgilenir. Bu nedenle, girişte ondalıktan ikiliye ve çıkışta ikiliden ondalığa dönüştürme zorunluluğu vardır. Çok fazla I/O'nun olduğu ve nispeten az, nispeten basit hesaplamaların olduğu uygulamalar için, sayıların ondalık formda depolanması ve çalıştırılması tercih edilir. Bu amaç için en yaygın gösterim, paketlenmiş ondalık sayıdır.

Çoğu makine, doğrudan paketlenmiş ondalık sayılar üzerinde işlem yapmak için aritmetik komutlar sağlar.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Örneğin, 246 için kod 0000 0010 0100 0110'dur. Bu kod, düz bir binary gösterime göre açıkça daha az kompakttır, ancak dönüştürme ek yükünü önler.



Characters



- Yaygın bir veri biçimi metin veya karakter dizileridir
- Karakter biçimindeki metin verileri, insanlar için çok uygun olmasına rağmen, veri işleme ve haberleşme sistemleri tarafından kolayca depolanamaz veya iletilemez.
 - Çünkü bu haberleşme sistemleri binary veriler için tasarlanmıştır
- Bu nedenle, karakterlerin bir dizi bit ile temsil edildiği bir dizi kod geliştirilmiştir.
 - Belki de bunun en eski yaygın örneği Mors kodudur.
- En yaygın kullanılan karakter kodu Uluslararası Referans Alfabesidir (IRA)
 - Amerika Birleşik Devletleri'nde ASCII olarak anılır
 - the American Standard Code for Information Interchange (ASCII)
- Karakterleri kodlamak için kullanılan diğer bir kod, Genişletilmiş İkili Kodlanmış Ondalık Değişim Kodudur (EBCDIC-Extended Binary Coded Decimal Interchange Code)
 - EBCDIC, IBM ana bilgisayarlarında kullanılır
 - 8 bitlik bir koddur. IRA'da olduğu gibi EBCDIC, paketlenmiş ondalık sayılarla uyumludur.
 - EBCDIC'de, 11110000'den 11111001'e kadar olan kodlar 0'dan 9'a kadar olan rakamları temsil eder.



Logical Data



- Her bir öge 0 veya 1 değerine sahip n tane 1-bitlik veri ögesinden oluşan bir n -bit birim
- Two advantages to bit-oriented view:
 - Bellek, her ögenin yalnızca 1 (true) ve 0 (false) değerlerini alabildiği bir dizi Boole veya binary veri ögelerini depolamak için en verimli şekilde kullanılabilir.
 - Bir veri ögesinin bitlerini değiştirmek için
 - Yazılımda kayan nokta işlemleri (*floating-point operations*) uygulanıyorsa, bazı işlemlerde önemli bitleri kaydırabilmemiz gerekir.
 - IRA'dan paketlenmiş ondalık sayıya dönüştürmek için, her baytın en sağdaki 4 bitini çıkarmamız gerekir.



Table 12.2

x86

Data Types

Data Type	Description
General	Byte, word (16 bits), doubleword (32 bits), quadword (64 bits), and double quadword (128 bits) locations with arbitrary binary contents.
Integer	A signed binary value contained in a byte, word, or doubleword, using twos complement representation.
Ordinal	An unsigned integer contained in a byte, word, or doubleword.
Unpacked binary coded decimal (BCD)	A representation of a BCD digit in the range 0 through 9, with one digit in each byte.
Packed BCD	Packed byte representation of two BCD digits; value in the range 0 to 99.
Near pointer	A 16-bit, 32-bit, or 64-bit effective address that represents the offset within a segment. Used for all pointers in a nonsegmented memory and for references within a segment in a segmented memory.
Far pointer	A logical address consisting of a 16-bit segment selector and an offset of 16, 32, or 64 bits. Far pointers are used for memory references in a segmented memory model where the identity of a segment being accessed must be specified explicitly.
Bit field	A contiguous sequence of bits in which the position of each bit is considered as an independent unit. A bit string can begin at any bit position of any byte and can contain up to 32 bits.
Bit string	A contiguous sequence of bits, containing from zero to $2_{32} - 1$ bits.
Byte string	A contiguous sequence of bytes, words, or doublewords, containing from zero to $2_{32} - 1$ bytes.
Floating point	See Figure 12.4.
Packed SIMD (single instruction, multiple data)	Packed 64-bit and 128-bit data types

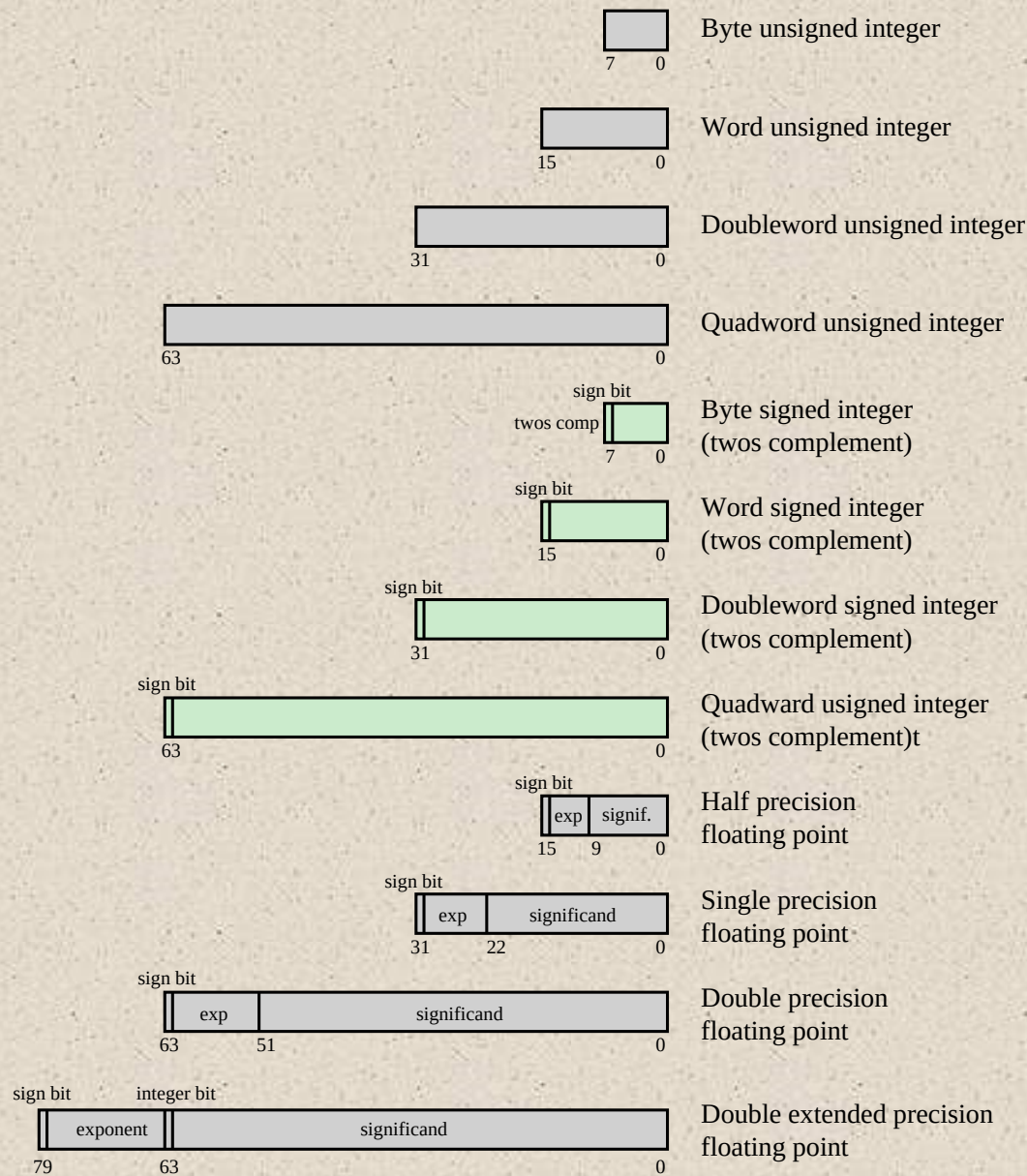
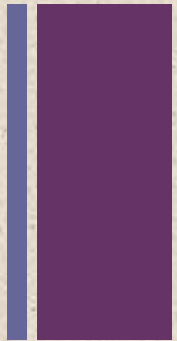


Figure 12.4 x86 Numeric Data Formats



Single-Instruction-Multiple-Data (SIMD) Data Types



- Multimedya uygulamalarının performansını optimize etmek için komut setinin uzantılarının bir parçası olarak x86 mimarisine eklendi
- Bu uzantılar arasında MMX (multimedya uzantıları) ve SSE (streaming SIMD uzantıları) bulunur
- Data types:
 - Packed byte and packed byte integer
 - Packed word and packed word integer
 - Packed doubleword and packed doubleword integer
 - Packed quadword and packed quadword integer
 - Packed single-precision floating-point and packed double-precision floating-point

ARM Data Types

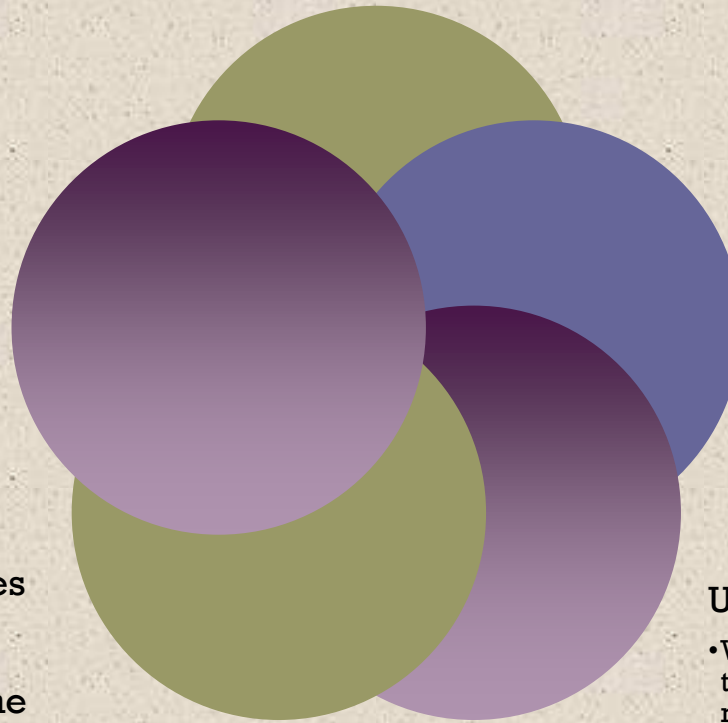


ARM processors support data types of:

- 8 (byte)
- 16 (halfword)
- 32 (word) bits in length

All three data types can also be used for two's complement signed integers

For all three data types an unsigned interpretation is supported in which the value represents an unsigned, nonnegative integer



Alignment checking

- When the appropriate control bit is set, a data abort signal indicates an alignment fault for attempting unaligned access

Unaligned access

- When this option is enabled, the processor uses one or more memory accesses to generate the required transfer of adjacent bytes transparently to the programmer

Sistem kontrol kaydındaki bir durum biti (E-bit), SETEND komutu kullanılarak program kontrolü altında setlenir ve sıfırlanır.

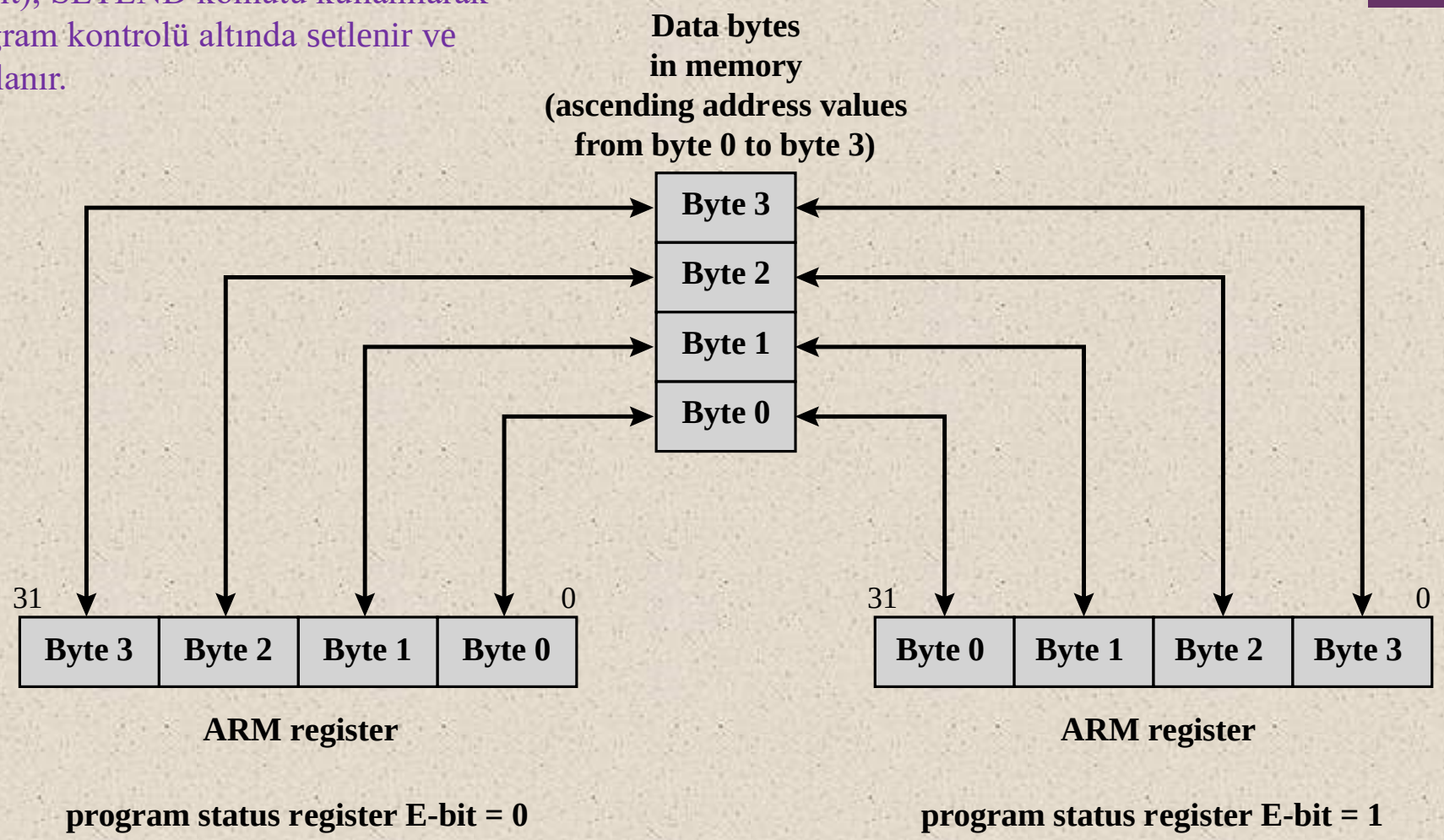


Figure 12.5 ARM Endian Support - Word Load/Store with E-bit



Table 12.3

Common Instruction Set Operations (page 1 of 2)

(Table can be found on page 426 in textbook.)

Type	Operation Name	Description
Data Transfer	Move (transfer)	Transfer word or block from source to destination
	Store	Transfer word from processor to memory
	Load (fetch)	Transfer word from memory to processor
	Exchange	Swap contents of source and destination
	Clear (reset)	Transfer word of 0s to destination
	Set	Transfer word of 1s to destination
	Push	Transfer word from source to top of stack
	Pop	Transfer word from top of stack to destination
Arithmetic	Add	Compute sum of two operands
	Subtract	Compute difference of two operands
	Multiply	Compute product of two operands
	Divide	Compute quotient of two operands
	Absolute	Replace operand by its absolute value
	Negate	Change sign of operand
	Increment	Add 1 to operand
	Decrement	Subtract 1 from operand
Logical	AND	Perform logical AND
	OR	Perform logical OR
	NOT (complement)	Perform logical NOT
	Exclusive-OR	Perform logical XOR
	Test	Test specified condition; set flag(s) based on outcome
	Compare	Make logical or arithmetic comparison of two or more operands; set flag(s) based on outcome
	Set Control Variables	Class of instructions to set controls for protection purposes, interrupt handling, timer control, etc.
	Shift	Left (right) shift operand, introducing constants at end
	Rotate	Left (right) shift operand, with wraparound end



Table 12.3

Common Instruction Set Operations (page 2 of 2)

(Table can be found on page 426 in textbook.)

Type	Operation Name	Description
Transfer of Control	Jump (branch)	Unconditional transfer; load PC with specified address
	Jump Conditional	Test specified condition; either load PC with specified address or do nothing, based on condition
	Jump to Subroutine	Place current program control information in known location; jump to specified address
	Return	Replace contents of PC and other register from known location
	Execute	Fetch operand from specified location and execute as instruction; do not modify PC
	Skip	Increment PC to skip next instruction
	Skip Conditional	Test specified condition; either skip or do nothing based on condition
	Halt	Stop program execution
	Wait (hold)	Stop program execution; test specified condition repeatedly; resume execution when condition is satisfied
	No operation	No operation is performed, but program execution is continued
Input/Output	Input (read)	Transfer data from specified I/O port or device to destination (e.g., main memory or processor register)
	Output (write)	Transfer data from specified source to I/O port or device
	Start I/O	Transfer instructions to I/O processor to initiate I/O operation
	Test I/O	Transfer status information from I/O system to specified destination
Conversion	Translate	Translate values in a section of memory based on a table of correspondences
	Convert	Convert the contents of a word from one form to another (e.g., packed decimal to binary)

Table 12.4

Processor Actions for Various Types of Operations

Data Transfer	Transfer data from one location to another
	If memory is involved: Determine memory address Perform virtual-to-actual-memory address transformation Check cache Initiate memory read/write
Arithmetic	May involve data transfer, before and/or after
	Perform function in ALU
	Set condition codes and flags
Logical	Same as arithmetic
Conversion	Similar to arithmetic and logical. May involve special logic to perform conversion
Transfer of Control	Update program counter. For subroutine call/return, manage parameter passing and linkage
I/O	Issue command to I/O module
	If memory-mapped I/O, determine memory-mapped address

(Table can be found on page 427 in textbook.)

Data Transfer



Her konum bellek, register
veya yığının tepesi olabilir.

En temel
makine
komutu türü



Belirtilmesi gerekenler:

Kaynak ve hedef operandların
konumu

Aktarılacak verilerin uzunluğu
belirtilmelidir

Her operand için adresleme modu
belirtilmelidir

Table 12.5

Examples of IBM EAS/390 Data Transfer Operations

Operation Mnemonic	Name	Number of Bits Transferred	Description
L	Load	32	Transfer from memory to register
LH	Load Halfword	16	Transfer from memory to register
LR	Load	32	Transfer from register to register
LER	Load (Short)	32	Transfer from floating-point register to floating-point register
LE	Load (Short)	32	Transfer from memory to floating-point register
LDR	Load (Long)	64	Transfer from floating-point register to floating-point register
LD	Load (Long)	64	Transfer from memory to floating-point register
ST	Store	32	Transfer from register to memory
STH	Store Halfword	16	Transfer from register to memory
STC	Store Character	8	Transfer from register to memory
STE	Store (Short)	32	Transfer from floating-point register to memory
STD	Store (Long)	64	Transfer from floating-point register to memory



- Çoğu makine toplama, çıkarma, çarpma ve bölme gibi temel aritmetik işlemleri sağlar
- Bunlar, işaretli tamsayı (sabit nokta- *fixed-point*) sayılar için sağlanmıştır
- Genellikle, kayan nokta (*floating-point*) ve paketlenmiş ondalık (*packed decimal*) sayılar için de sağlanırlar
- Diğer olası işlemler, çeşitli tek operandlı komutları içerir:
 - **Absolute**
 - Operandın mutlak değerini alır
 - **Negate**
 - Operandın tersini alır
 - **Increment**
 - Operanda 1 ekler
 - **Decrement**
 - Operand'dan 1 çıkarır



Arithmetic

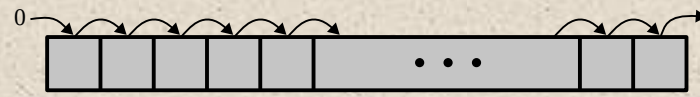


Table 12.6

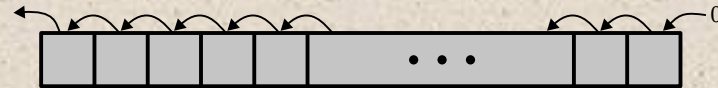
Basic Logical Operations

P	Q	NOT P	P AND Q	P OR Q	P XOR Q	P=Q
0	0	1	0	0	0	1
0	1	1	0	1	1	0
1	0	0	0	1	1	0
1	1	0	1	1	0	1

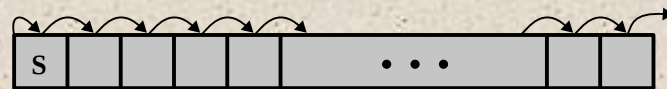
EQUAL is a useful
binary test.



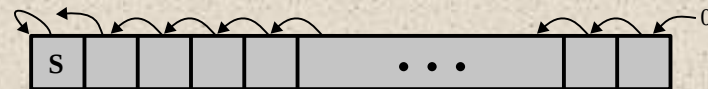
(a) Logical right shift



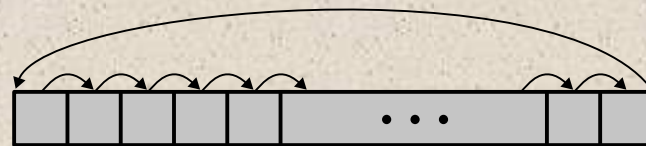
(b) Logical left shift



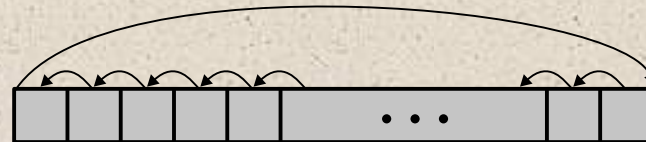
(c) Arithmetic right shift



(d) Arithmetic left shift



(e) Right rotate



(f) Left rotate

Figure 12.6 Shift and Rotate Operations

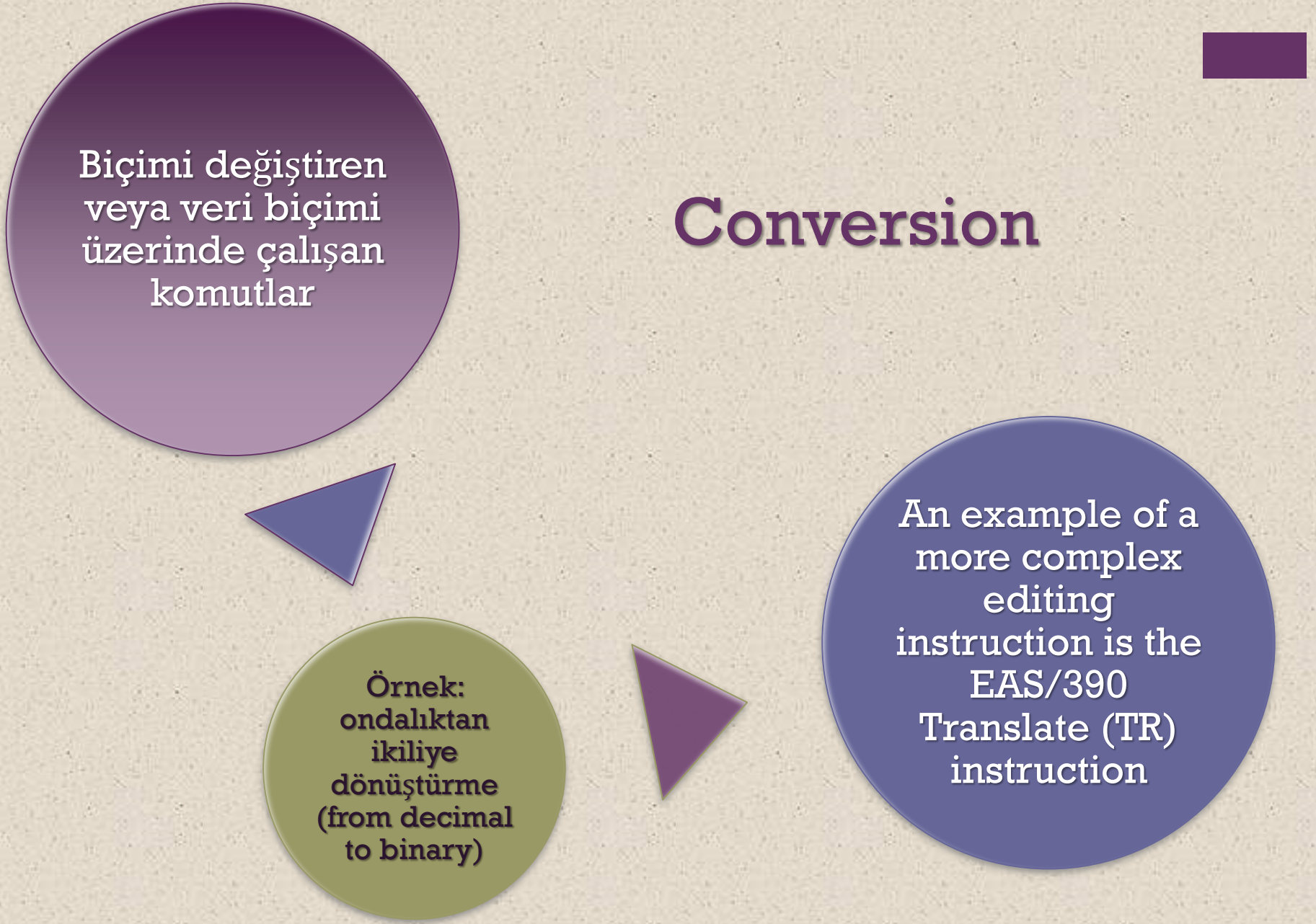


Table 12.7

Examples of Shift and Rotate Operations

Input	Operation	Result
10100110	Logical right shift (3 bits)	00010100
10100110	Logical left shift (3 bits)	00110000
10100110	Arithmetic right shift (3 bits)	11110100
10100110	Arithmetic left shift (3 bits)	10110000
10100110	Right rotate (3 bits)	11010100
10100110	Left rotate (3 bits)	00110101

Aritmetik kaydırma işlemi, verileri işaretli bir tam sayı olarak ele alır ve işaret bitini kaydırmaz.



Biçimi değiştiren
veya veri biçimi
üzerinde çalışan
komutlar

Conversion

Örnek:
ondalıktan
ikiliye
dönüştürme
(from decimal
to binary)

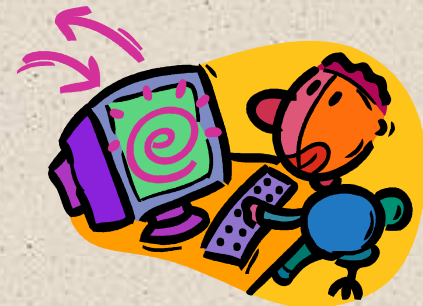
An example of a
more complex
editing
instruction is the
EAS/390
Translate (TR)
instruction



Input/Output



- Çeşitli yaklaşımlar:
 - Isolated programmed I/O
 - Memory-mapped programmed I/O
 - DMA
 - Use of an I/O processor
- Çoğu uygulama, parametreler, kodlar veya komut kelimeleriyle belirtilen belirli eylemlerle yalnızca birkaç I/O komutu sağlar.



System Control

Sadece işlemci belirli bir ayrıcalıklı durumdayken (*privileged state*) veya özel bir ayrıcalıklı bellek alanında bir programı çalıştırırken yürütülebilen komutlar

Genellikle bu komutlar işletim sisteminin kullanımı için ayrılmıştır.

Sistem kontrol işlemlerine örnekler:

Bir sistem kontrol komutu, bir kontrol registerini okuyabilir veya değiştirebilir

Bir depolama koruma anahtarını (*storage protection key*) okuma veya değiştirme komutu

Çoklu programlama sistemindeki (*multiprogramming system*) proses kontrol bloklarına erişim

+ Transfer of Control

- Kontrol transferi (*transfer-of-control*) işlemlerinin gerekli olmasının nedenleri:
 - Her bir komutu birden fazla kez yürütebilmek esastır.
 - Hemen hemen tüm programlar bazı karar vermeler içerir
 - Görevi teker teker üzerinde çalışılabilecek daha küçük parçalara ayırmak için mekanizmalar varsa buna yardımcı olur.
- Komut setlerinde bulunan en yaygın kontrol transferi işlemleri:
 - Branch
 - Skip
 - Procedure call

Bir dallanma komutu (branch instruction), aynı zamanda bir atlama (jump) komutu olarak da adlandırılır, operandlarından biri olarak yürütülecek sonraki komutun adresini içerir. Çoğu zaman, komut bir koşullu dallanma (**conditional branch**) komutudur Yani, yalnızca belirli bir koşul karşılanırsa dallanma yapılır (program sayacını operand'da belirtilen adrese eşitleyerek günceller). Aksi takdirde, sıradaki bir sonraki komut yürütülür (her zamanki gibi program sayacını artırır).

Dallanmanın her zaman yapıldığı bir dallanma komutu, bir koşulsuz dallanmadır (**unconditional branch**).

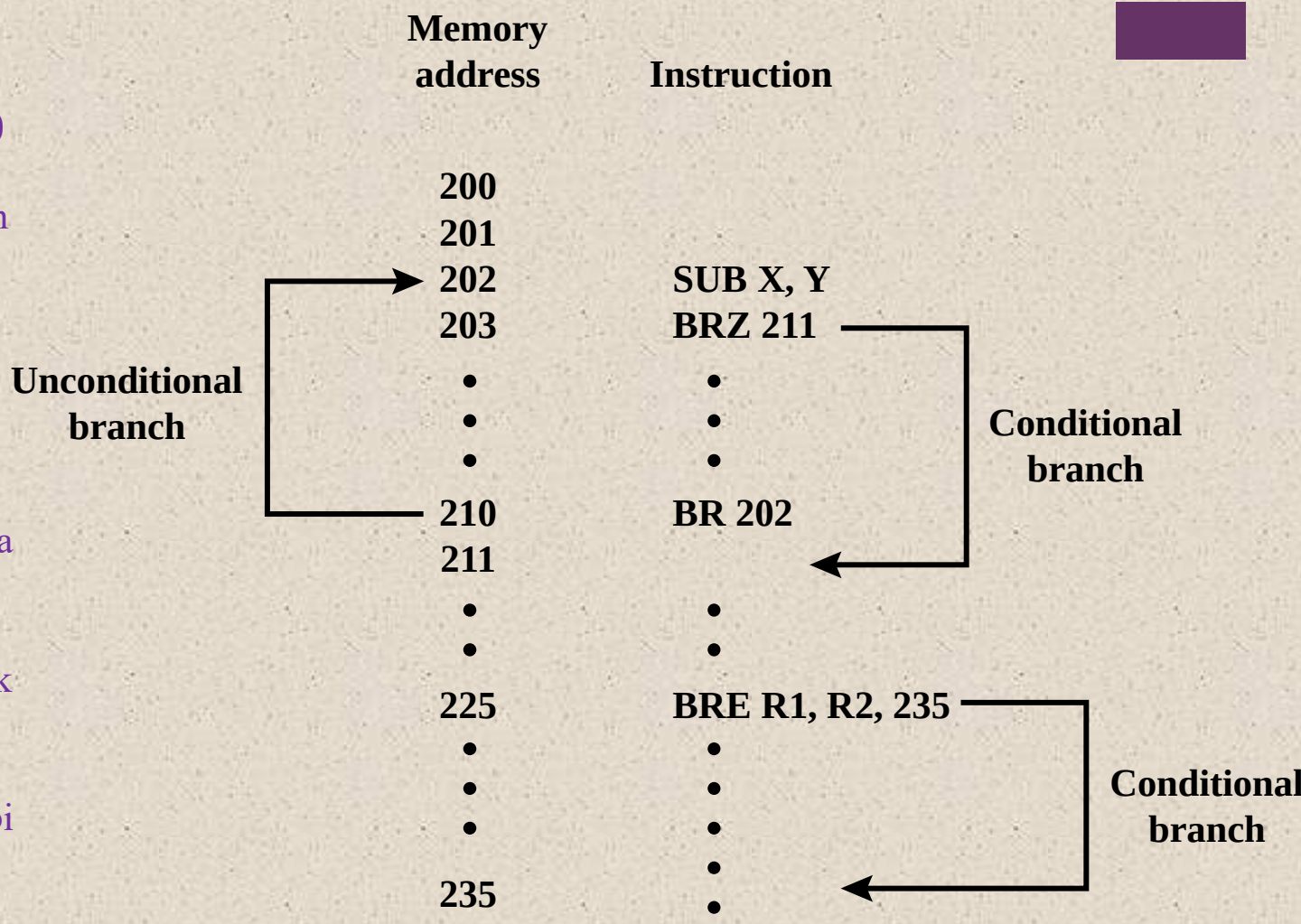


Figure 12.7 Branch Instructions

Skip Instructions

Kontrol transferi komutunun başka bir biçimi de «skip» komutudur.

Örtülü/imalı bir adres içerir (implied address)

Tipik olarak, bir komutun atlanacağını ima eder, dolayısıyla zımnî adres, bir sonraki komutun adresi artı bir komut uzunluğuna eşittir.

Atlama komutu (*skip instruction*) bir hedef adres alanı gerektirmediğinden, başka işler yapmak serbesttir.

Örnek bir komut «increment-and-skip-if-zero (ISZ) instruction»



Procedure Call Instructions



- Daha büyük bir programa dahil edilmiş bağımsız bilgisayar programı
 - Programın herhangi bir noktasında prosedür çağrılabilir (*invoked*, or *called*)
 - İşlemciye gidip tüm prosedürü yürütmesi ve ardından çağrının gerçekleştiği noktaya dönmesi komutu verilir.
- Prosedürlerin kullanılmasının iki ana nedeni:
 - Ekonomi
 - Bir prosedür, aynı kod parçasının birçok kez kullanılmasına izin verir
 - Modülerlik
- İki temel komutu içerir:
 - Mevcut konumdan prosedüre dallanan bir çağrı komutu
 - Prosedürden çağrıldığı yere geri dönen komut

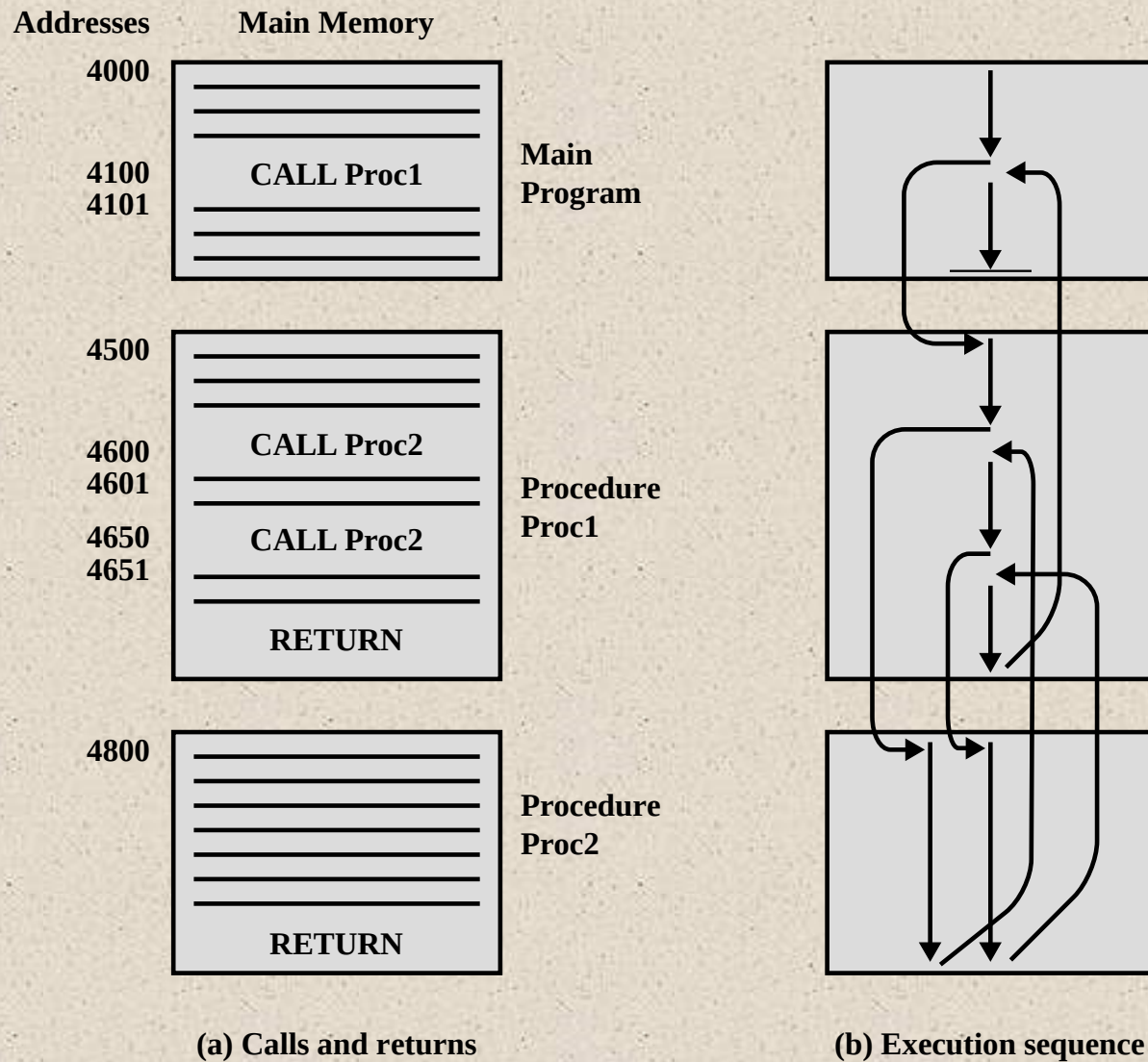


Figure 12.8 Nested Procedures

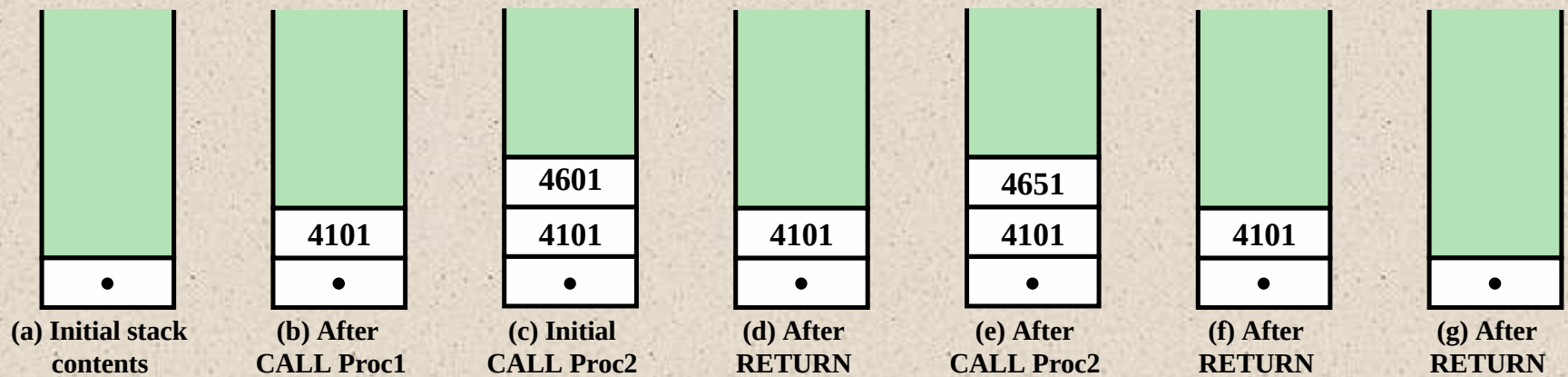
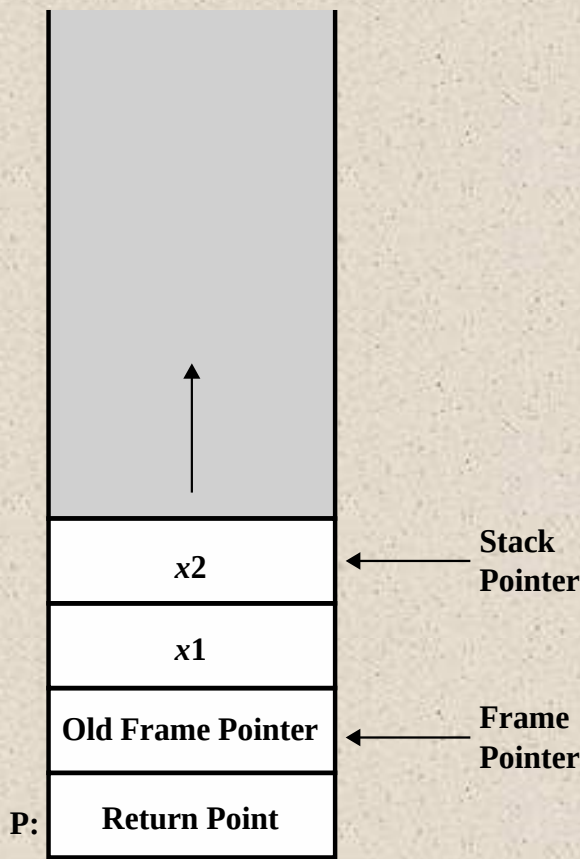
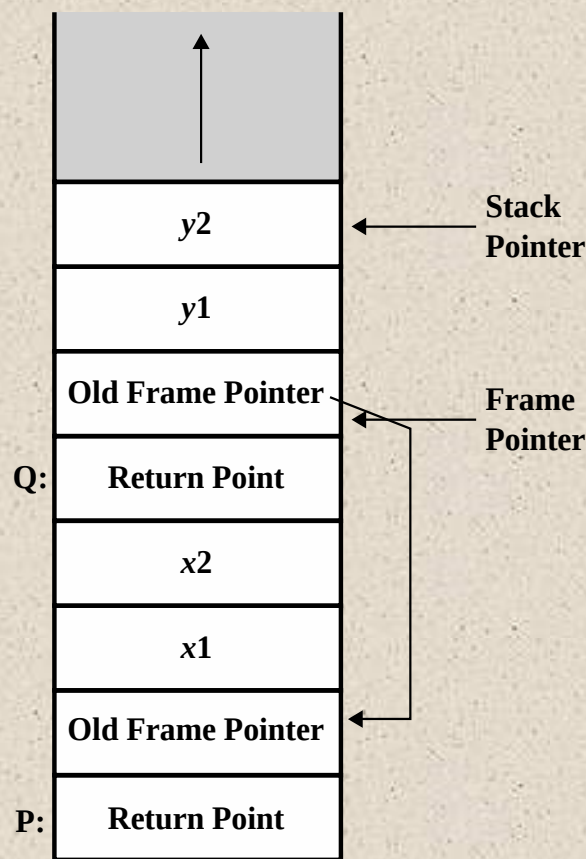


Figure 12.9 Use of Stack to Implement Nested Procedures of Figure 12.8



(a) P is active



(b) P has called Q

Figure 12.10 Stack Frame Growth Using Sample Procedures P and Q

Yığın, parametre geçişine (parameter passing) daha esnek bir yaklaşım sağlar. İşlemci bir çağrıyı yürüttüğünde, yalnızca dönüş adresini yığına itmekle kalmaz, çağrılan prosedüre geçirilecek parametreleri de yığına iter. Çağrılan prosedür, yığından parametrelere erişebilir. Döndükten sonra, dönüş parametreleri de yığına yerleştirilebilir. Bir prosedür çağrısı için saklanan, dönüş adresi dahil olmak üzere tüm parametreler kümesine yığın çerçevesi (*stack frame*) adı verilir.

+ x86 Operation Types

- X86, bir dizi özel komut içeren karmaşık bir işlem türü dizisi sağlar
- Amaç, derleyiciyi yazan kişiye yüksek seviyeli dil programlarının optimize edilmiş makine dili çevirisini üretmesi için araçlar sağlamaktır.
- Prosedür çağırısı / dönüşü (call/return) desteklemek için dört komut sağlar
 - CALL
 - ENTER
 - LEAVE
 - RETURN
- Yeni bir prosedür çağırıldığında, yeni prosedüre girişte aşağıdakiler gerçekleştirilmelidir:
 - Dönüş notası yığına itilir
 - Aktif çerçeve işaretçisi yığına itilir
 - Yığın işaretçisi çerçeve işaretçisinin yeni değeri olarak kopyalanır (*Copy the stack pointer as the new value of the frame pointer.*)
 - Çerçeve tahsisi için yığın işaretçisi ayarlanır

Table 12.8

x86 Status Flags

Status Bit	Name	Description
CF	Carry	Indicates carrying or borrowing out of the left-most bit position following an arithmetic operation. Also modified by some of the shift and rotate operations.
PF	Parity	Parity of the least-significant byte of the result of an arithmetic or logic operation. 1 indicates even parity; 0 indicates odd parity.
AF	Auxiliary Carry	Represents carrying or borrowing between half-bytes of an 8-bit arithmetic or logic operation. Used in binary-coded decimal arithmetic.
ZF	Zero	Indicates that the result of an arithmetic or logic operation is 0.
SF	Sign	Indicates the sign of the result of an arithmetic or logic operation.
OF	Overflow	Indicates an arithmetic overflow after an addition or subtraction for twos complement arithmetic.



Table 12.9

x86 Condition Codes for Conditional Jump and SETcc Instructions

Symbol	Condition Tested	Comment
A, NBE	CF=0 AND ZF=0	Above; Not below or equal (greater than, unsigned)
AE, NB, NC	CF=0	Above or equal; Not below (greater than or equal, unsigned); Not carry
B, NAE, C	CF=1	Below; Not above or equal (less than, unsigned); Carry set
BE, NA	CF=1 OR ZF=1	Below or equal; Not above (less than or equal, unsigned)
E, Z	ZF=1	Equal; Zero (signed or unsigned)
G, NLE	[(SF=1 AND OF=1) OR (SF=0 and OF=0)] AND [ZF=0]	Greater than; Not less than or equal (signed)
GE, NL	(SF=1 AND OF=1) OR (SF=0 AND OF=0)	Greater than or equal; Not less than (signed)
L, NGE	(SF=1 AND OF=0) OR (SF=0 AND OF=1)	Less than; Not greater than or equal (signed)
LE, NG	(SF=1 AND OF=0) OR (SF=0 AND OF=1) OR (ZF=1)	Less than or equal; Not greater than (signed)
NE, NZ	ZF=0	Not equal; Not zero (signed or unsigned)
NO	OF=0	No overflow
NS	SF=0	Not sign (not negative)
NP, PO	PF=0	Not parity; Parity odd
O	OF=1	Overflow
P	PF=1	Parity; Parity even
S	SF=1	Sign (negative)

(Table can be found on page 440 in the textbook.)



Table 12.10

MMX

Instruction Set

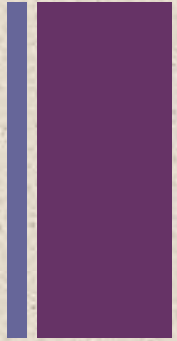
Category	Instruction	Description
Arithmetic	PADD [B, W, D]	Parallel add of packed eight bytes, four 16-bit words, or two 32-bit doublewords, with wraparound.
	PADDs [B, W]	Add with saturation.
	PADDUS [B, W]	Add unsigned with saturation
	PSUB [B, W, D]	Subtract with wraparound.
	PSUBS [B, W]	Subtract with saturation.
	PSUBUS [B, W]	Subtract unsigned with saturation
	PMULHW	Parallel multiply of four signed 16-bit words, with high-order 16 bits of 32-bit result chosen.
	PMULLW	Parallel multiply of four signed 16-bit words, with low-order 16 bits of 32-bit result chosen.
Comparison	PMADDWD	Parallel multiply of four signed 16-bit words; add together adjacent pairs of 32-bit results.
	PCMPEQ [B, W, D]	Parallel compare for equality; result is mask of 1s if true or 0s if false.
	PCMPGT [B, W, D]	Parallel compare for greater than; result is mask of 1s if true or 0s if false.
Conversion	PACKUSWB	Pack words into bytes with unsigned saturation.
	PACKSS [WB, DW]	Pack words into bytes, or doublewords into words, with signed saturation.
	PUNPCKH [BW, WD, DQ]	Parallel unpack (interleaved merge) high-order bytes, words, or doublewords from MMX register.
	PUNPCKL [BW, WD, DQ]	Parallel unpack (interleaved merge) low-order bytes, words, or doublewords from MMX register.
Logical	PAND	64-bit bitwise logical AND
	PNDN	64-bit bitwise logical AND NOT
	POR	64-bit bitwise logical OR
	PXOR	64-bit bitwise logical XOR
Shift	PSLL [W, D, Q]	Parallel logical left shift of packed words, doublewords, or quadword by amount specified in MMX register or immediate value.
	PSRL [W, D, Q]	Parallel logical right shift of packed words, doublewords, or quadword.
	PSRA [W, D]	Parallel arithmetic right shift of packed words, doublewords, or quadword.
Data Transfer	MOV [D, Q]	Move doubleword or quadword to/from MMX register.
State Mgt	EMMS	Empty MMX state (empty FP registers tag bits).

(Table can be found on page 442 in the textbook.)

Note: If an instruction supports multiple data types [byte (B), word (W), doubleword (D), quadword (Q)], the data types are indicated in brackets.



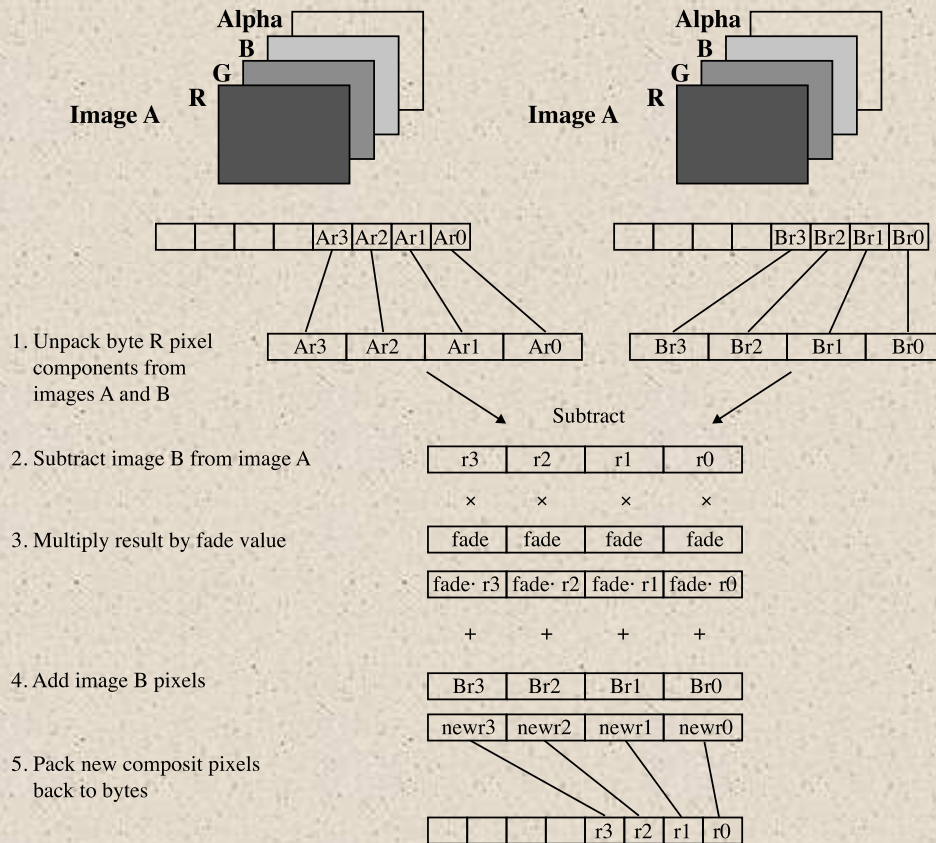
x86 Single-Instruction, Multiple-Data (SIMD) Instructions



- 1996 Intel, MMX teknolojisini Pentium ürün serisine ekledi
 - MMX, multimedya görevleri için son derece optimize edilmiş bir komutlar kümesidir
- Video ve ses verileri tipik olarak büyük boyutlardaki küçük veri türleri dizilerinden oluşur
- MMX'te üç yeni veri türü tanımlanmıştır
 - Packed byte
 - Packed word
 - Packed doubleword
- Her veri türü 64 bit uzunluğundadır ve her biri sabit noktalı bir tamsayı (*fixed-point integer*) tutan çok sayıda küçük veri alanından oluşur

1996 yılında Intel, MMX teknolojisini Pentium ürün serisine dahil etti. MMX, multimedya görevleri için son derece optimize edilmiş komutlar setidir. Verileri SIMD (tek komut, çoklu veri) biçiminde işleyen 57 yeni komut vardır, bu da toplama veya çarpma gibi aynı işlemi aynı anda birden fazla veri ögesi üzerinde gerçekleştirmeyi mümkün kılar. Her komutun yürütülmesi tipik olarak tek bir saat döngüsü alır.

Doğru uygulama için, bu hızlı paralel işlemler, MMX komutlarını kullanmayan kıyaslanabilen algoritmalara göre **iki ila sekiz kat** hız sağlayabilir.



MMX code sequence performing this operation:

```

pxor      mm7, mm7      ;zero out mm7
movq      mm3, fad_val  ;load fade value replicated 4 times
movd      mm0, imageA   ;load 4 red pixel components from image A
movd      mm1, imageB   ;load 4 red pixel components from image B
punpckblw mm0, mm7      ;unpack 4 pixels to 16 bits
punpckblw mm1, mm7      ;unpack 4 pixels to 16 bits
psubw     mm0, mm1      ;subtract image B from image A
pmulhw    mm0, mm3      ;multiply the subtract result by fade values
paddw     mm0, mm1      ;add result to image B
packuswb  mm0, mm7      ;pack 16-bit results back to bytes

```

Figure 12.11 Image Compositing on Color Plane Representation

Şekil 12.11, bir piksel seti için gereken adımların sırasını göstermektedir. 8-bit piksel bileşenleri, MMX 16-bit çarpma yeteneğini barındırmak için 16-bit elemanlara dönüştürülür. Bu görüntüler 640 * 480 çözünürlük kullanıyorsa ve çözünme tekniği, solma (*fade*) değerinin tüm 255 olası değerini kullanıyorsa, MMX kullanılarak yürütülen toplam komut sayısı 535 milyondur. MMX komutları olmadan gerçekleştirilen aynı hesaplama, 1.4 milyar komut yürütme gerektirir [INTE98].

ARM Operation Types

Load and store
instructions

Branch
instructions

Data-processing
instructions

Multiply
instructions

Parallel addition
and subtraction
instructions

Extend
instructions

Status register
access
instructions



Table 12.11

**ARM
Conditions
for
Conditional
Instruction
Execution**

(Table can be found on
Page 445 in the textbook.)

Code	Symbol	Condition Tested	Comment
0000	EQ	$Z = 1$	Equal
0001	NE	$Z = 0$	Not equal
0010	CS/HS	$C = 1$	Carry set/unsigned higher or same
0011	CC/LO	$C = 0$	Carry clear/unsigned lower
0100	MI	$N = 1$	Minus/negative
0101	PL	$N = 0$	Plus/positive or zero
0110	VS	$V = 1$	Overflow
0111	VC	$V = 0$	No overflow
1000	HI	$C = 1 \text{ AND } Z = 0$	Unsigned higher
1001	LS	$C = 0 \text{ OR } Z = 1$	Unsigned lower or same
1010	GE	$N = V$ $[(N = 1 \text{ AND } V = 1)$ $\text{OR } (N = 0 \text{ AND } V = 0)]$	Signed greater than or equal
1011	LT	$N \neq V$ $[(N = 1 \text{ AND } V = 0)$ $\text{OR } (N = 0 \text{ AND } V = 1)]$	Signed less than
1100	GT	$(Z = 0) \text{ AND } (N = V)$	Signed greater than
1101	LE	$(Z = 1) \text{ OR } (N \neq V)$	Signed less than or equal
1110	AL	—	Always (unconditional)
1111	—	—	This instruction can only be executed unconditionally

+ Summary

Chapter 12

- Machine instruction characteristics
 - Elements of a machine instruction
 - Instruction representation
 - Instruction types
 - Number of addresses
 - Instruction set design
- Types of operands
 - Numbers
 - Characters
 - Logical data

Instruction Sets: Characteristics and Functions

- Intel x86 and ARM data types
- Types of operations
 - Data transfer
 - Arithmetic
 - Logical
 - Conversion
 - Input/output
 - System control
 - Transfer of control
- Intel x86 and ARM operation types